MegaVault AI Powered E-Commerce Shopping Platform

Submitted By

- N. Sai Teja
- B. Harsha Vardhan Babu
- J. Bhargava Aditya
- CH. Tejas Kumar

Table of Contents

1.Introduction

MegaVault is a full-stack AI powered e-commerce shopping platform developed during a company internship. The project focuses on building a secure, responsive, scalable, and user-friendly online shopping system.

The application allows customers to browse products, search using keywords, apply price-based filters, view product categories, add items to cart, maintain wishlist items, and authenticate using Google login. It also provides an admin dashboard for secure catalog management.

From a technical perspective, MegaVault follows a layered architecture. The frontend is developed using React.js, while backend services are implemented using Spring Boot. The backend communicates with a MySQL database through Spring Data JPA and Hibernate. Security is implemented using Spring Security, BCrypt password hashing, Google authentication, and role-based access control.

2.Problem Statement

E-commerce platforms require quick product discovery, secure user access, structured product management, and smooth shopping workflows. Customers may face difficulty finding suitable products when catalogs contain multiple categories and large product lists. Manual filtering can become time-consuming and less efficient.

Administrators also require secure access to manage product details, categories, SKU information, and catalog operations. Without role-based control, unauthorized access may lead to incorrect product updates, accidental deletion, or catalog inconsistency.

MegaVault addresses these challenges by providing AI-assisted product discovery, category-based browsing, INR price filtering, cart and wishlist features, Google authentication, admin dashboard access, and audit-friendly product deletion responses.

3.Objectives

The main objectives of the internship project are:

- To develop a responsive e-commerce frontend using React.js.
- To build RESTful backend services using Spring Boot.
- To implement AI-assisted product discovery.
- To support product search using keywords, categories, and price limits.
- To organize products into structured departments.
- To implement shopping cart and wishlist workflows.
- To integrate Google authentication for secure sign-in.
- To implement Spring Security and BCrypt password hashing.
- To apply role-based access control for customer and admin operations.
- To connect the backend with MySQL using Spring Data JPA/Hibernate.
- To provide audit-friendly product deletion responses.
- To create a scalable foundation for future e-commerce enhancements.

4.Scope of the Internship Project

The scope of MegaVault includes customer-facing shopping features, backend API development, database persistence, authentication, authorization, admin catalog operations, and AI-assisted product search.

Included Features

- Customer home page
- Product catalog display
- Category-based product browsing
- AI-powered search assistance
- Product search by keyword
- Product filtering by price
- Cart management
- Wishlist management
- Google authentication
- Admin dashboard
- Product and category management
- Role-based access control
- REST API integration

- MySQL database storage
- Product deletion audit response

Future Extendable Areas

- Payment gateway integration
- Complete checkout process
- Order tracking
- Personalized recommendations
- Inventory management
- Analytics dashboard
- Cloud deployment
- CI/CD pipeline

5.Technology Stack

Layer/Component	Technology Used
Frontend	React.js/React 18
Backend	Spring Boot 3.2.3
Programming Language	Java 17 LTS
Database	MySQL 8.0
ORM Framework	Spring Data JPA/Hibernate
Security Framework	Spring Security
Password Hashing	BCrypt
Authentication	Google Authentication
API Communication	REST API with HTTP/JSON
UI Styling	Glassmorphism CSS
Currency Format	Indian Rupee ₹

6.System Architecture

MegaVault follows a layered client-server architecture. The frontend sends HTTP requests to backend REST controllers. The backend processes business logic through service classes, applies security rules, interacts with repositories, and stores or retrieves data from MySQL.

Architecture Flow

React.js Web Client → REST Controllers → Service Layer → Security Layer → Spring Data JPA/Hibernate → MySQL Database

Architecture Layers

Layer	Description
Frontend Layer	Handles product display, search, cart, wishlist, authentication screens, and admin dashboard
Controller Layer	Receives API requests and returns JSON responses
Service Layer	Processes business logic, validation, filtering, and catalog operations
Security Layer	Handles authentication, authorization, password hashing, and role checks
Repository Layer	Performs database operations using Spring Data JPA
Database Layer	Stores users, products, categories, orders, and related records

Benefits of Architecture

- Clear separation of concerns
- Improved maintainability
- Scalable frontend-backend communication

7.Functional Modules

7.1 Customer Home

The customer home page is the main entry point of the platform. It displays products, product categories, product cards, search options, cart access, wishlist access, and navigation to product details.

7.2 Product Catalog

The product catalog displays available products with important details such as product name, price, rating, category, image, SKU, and description.

7.3 Product Search and Filtering

The search module allows products to be found using keywords, categories, and price-based conditions. Search functionality improves catalog usability and reduces manual browsing time.

7.4 Category Management

Products are organized into the following departments:

- Audio
- Wearables
- Gaming
- Electronics
- Fashion
- Smart Home

Category management supports structured browsing and department-level administration.

7.5 Shopping Cart

The cart module supports:

- Adding products to cart
- Updating product quantity

- Removing products from cart

7.6 Wishlist

The wishlist module allows products to be saved for future purchase planning and comparison. It improves shopping flexibility by separating saved items from cart items.

7.7 Google Authentication

Google authentication provides a familiar and secure sign-in process. After identity verification, MegaVault applies internal role-based permissions for customer and admin access.

7.8 Admin Dashboard

The admin dashboard provides secure access to catalog management features. It supports product management, category control, SKU-related operations, catalog review, and product deletion with audit response.

8.AI Powered Product Discovery

MegaVault includes AI-assisted product discovery to improve search efficiency. The AI-oriented search helps interpret natural shopping queries and identify product intent, category intent, and price constraints.

Example Queries

- Headphones under ₹5,000
- Products under ₹10,000
- Gaming accessories below ₹20,000
- Smart home devices under ₹15,000
- Find wearable products

Feature	Description
Smart Search	Finds products using meaningful query keywords
Price-Aware Filtering	Detects price limits from search queries

Category Intent Detection	Identifies relevant departments from user input

9.REST API Design

MegaVault uses REST APIs for communication between the React.js frontend and Spring Boot backend. API responses are returned in JSON format.

Method	Endpoint	Description
GET	/api/products	Retrieves the complete product catalog
GET	/api/products/category/{cat}	Retrieves products by category
GET	/api/products/search?q={query}	Searches products using query text
POST	/api/products	Creates a new product
DELETE	/api/products/{id}	Deletes a product and returns audit details
POST	/api/auth/register	Handles registration/authentication workflow

Product Deletion Audit Response

The delete API returns:

- Status
- Message
- Deletion timestamp
- Deleted product ID
- Product title
- SKU
- Category
- Price

This response helps administrators confirm which product was removed and supports audit-friendly tracking.

10.Database Design

MegaVault uses MySQL 8.0 for relational data storage. Spring Data JPA and Hibernate are used to map Java objects to database tables and simplify persistence operations.

Main Database Tables

Table	Purpose
USERS	Stores user account details, email, password hash, Google reference, verification status, role, and timestamps
PRODUCTS	Stores product ID, SKU, title, price, rating, category ID, image URL, description, stock status, and timestamps
CATEGORIES	Stores category ID, category name, description, and department type
ORDERS	Stores order ID, order number, customer ID, total amount, order status, created timestamp, and updated timestamp

Database Purpose

The database maintains structured records for users, products, categories, and order-related information. Foreign key relationships and JPA entity mappings help maintain consistency between related tables.

11.Security and Role-Based Access Control

MegaVault implements backend security using Spring Security and BCrypt. Passwords are protected using BCrypt hashing. Access to customer and admin features is controlled using assigned roles.

User Roles

Role	Access Permissions
ROLE_CUSTOMER	Product browsing, search, cart, and wishlist
ROLE_CATEGORY_ADMIN	Department-level product and SKU management
ROLE_SUPER_ADMIN	System-wide administrative control

Security Features

- Google authentication
- BCrypt password hashing
- Protected backend resources
- Role-based API access
- Admin operation restrictions
- Secure product and category management

Authentication confirms user identity, while authorization determines allowed operations based on role.

12.User Interface Design

MegaVault uses a modern glassmorphism-based interface with a responsive layout. The UI is designed for desktop, tablet, and mobile screens.

Theme Colors

Purpose	Color Code
Primary Orange	#F97316
Hover Orange	#EA580C
Sky Blue	#0EA5E9
Gold	#F59E0B
Dark Background	#0B0F17
Light Background	#F8FAFC

Main UI Components

- Home page
- Product cards
- Search bar
- Category panels
- Cart page
- Wishlist page
- Google authentication screen
- Admin dashboard
- AI assistant interface

The interface focuses on clear navigation, quick product discovery, readable product details, and smooth shopping actions.

13.Testing and Validation

Testing ensures that the system works correctly across frontend, backend, security, and database layers.

Testing Areas

- Product catalog retrieval
- Category filtering
- AI-assisted search processing
- Price filtering
- Product creation
- Product deletion response
- Cart operations
- Wishlist operations
- Google authentication
- Role-based access restrictions
- API status code verification
- JSON response validation
- Responsive UI testing
- Frontend error handling

Validation Outcome

Successful validation confirms that MegaVault supports stable product browsing, secure access, reliable API communication, structured data persistence, and controlled administrative operations.

14.Deployment and Execution

Local Execution Steps

1. Install Java 17 or later.
2. Install and configure MySQL 8.0.
3. Create the project database, such as megavault_db.
4. Configure Spring Boot database properties.
5. Start the Spring Boot backend server.
6. Install frontend dependencies.
7. Run the React.js frontend application.
8. Access the application through a browser.
9. Test customer and admin workflows.

Execution Requirement

Spring Boot 3.2.3 requires Java 17 or later. MySQL database credentials must match the backend configuration.

15.Internship Learning Outcomes

The MegaVault internship project provided practical exposure to:

- Full-stack web application development
- React.js component-based frontend design
- Spring Boot REST API development
- Java backend programming
- MySQL relational database management
- JPA/Hibernate entity mapping
- Authentication and authorization workflows
- Spring Security implementation
- Google authentication integration
- Role-based access control
- Responsive UI design
- API testing and validation
- Admin dashboard implementation
- Audit-friendly backend response design

16.Achievements

The project successfully achieved the following:

- Developed a responsive React.js shopping interface.
- Built Spring Boot REST backend services.
- Integrated MySQL database using JPA/Hibernate.
- Implemented product and category management.
- Added AI-assisted product discovery.
- Supported keyword and price-based product search.
- Implemented cart and wishlist workflows.
- Integrated Google authentication.
- Applied Spring Security and BCrypt.
- Implemented role-based access control.
- Created admin dashboard functionality.
- Added audit-friendly product deletion response.
- Designed a modern glassmorphism-based UI.

17.Future Scope

MegaVault can be enhanced with the following future features:

- Payment gateway integration
- Checkout workflow
- Order tracking
- Customer notifications
- Personalized product recommendation engine
- Analytics and reporting dashboard
- Cloud deployment and auto-scaling
- Automated unit testing
- Integration testing
- End-to-end testing
- CI/CD pipeline
- Production monitoring
- Persistent audit-log storage
- Inventory management
- Coupon and discount management
- Customer reviews and ratings

18.Conclusion

MegaVault AI Powered E-Commerce Shopping Platform is a company internship project that demonstrates the development of a secure, scalable, and responsive full-stack e-commerce application. The project combines React.js frontend development, Spring Boot backend services, Java programming, MySQL database storage, JPA/Hibernate persistence, Google authentication, Spring Security, and role-based access control.

The system supports important e-commerce workflows such as product browsing, AI-assisted search, category filtering, cart management, wishlist management, admin dashboard operations, and audit-friendly product deletion. The project also provides a strong technical foundation for future enhancements such as payment integration, order tracking, analytics, recommendation systems, cloud deployment, and production monitoring.

19.References

1. Spring Boot 3.2.3 Reference Documentation
2. React.js Documentation
3. MySQL 8.0 Documentation
4. Spring Security Documentation
5. Spring Data JPA/Hibernate Documentation
6. Java 17 LTS Documentation
7. Gemini AI References